

A High-Level Correctness Condition for Reagent Execution

Working note - seeds for meeting with Prof. Kartik

April 24, 2026

Abstract

We propose a correctness condition for the concurrent execution of reagents à la Turon [1, 2], building on the transactional memory semantics tradition of Guerraoui–Kapalka [3], Doherty–Groves–Luchangco–Moir–Scott [4], and, in particular, Attiya–Gotsman’s client-observable framework [5]. The condition generalises *opacity* to account for two features of reagents that ordinary STM does not have: external choice via (+) and synchronous rendezvous via channel swap. We state the condition as a history-level predicate and sketch what it obliges of the layered implementations in `01_kcas-06_reagent_cps`.

1 Introduction

Reagents are composable atomic actions. A reagent $r : \text{Reagent}(A, B)$ transforms an input of type A into an output of type B while effecting an atomic state transition on shared memory and, possibly, on synchronous channels. Reagents compose in three ways:

- *sequencing* $r_1 \gg r_2$: the two reagents commit as one atomic step;
- *choice* $r_1 + r_2$: an external (non-deterministic) choice between two alternatives, only one of which is committed;
- *rendezvous swap* ep : a two-party synchronous action pairing with a dual swap on the opposite endpoint.

Classical STM correctness conditions — opacity in particular — pin down the intuition that “each committed transaction appears to execute instantaneously”. They do not, out of the box, cover (i) the fact that a reagent with a (+) at its root has a choice of *which* committed action the linearisation sees, nor (ii) the fact that a swap’s linearisation point coincides, by construction, with that of its dual.

We aim to state a single condition, *reagent opacity*, that captures both. The condition is history-based and can be checked without reference to any particular implementation; in the layered codebase it yields concrete proof obligations for each layer.

2 Background: Transactional Opacity

Histories. Following Herlihy–Wing and the Attiya–Gotsman [5] presentation, a *history* H is a finite sequence of *events* of the form $\langle p, e \rangle$, where p is a thread identifier and e is one of

$$\text{inv}(r, v), \quad \text{res}(r, w), \quad \text{ret}(r),$$

respectively invoking reagent r with input v , responding with output w , or aborting/retrying before any commit. Invocations are paired with responses or aborts in program order within each thread. The projection of H to thread p is written $H|p$.

Sequential histories. A history is *sequential* if no two invocations are pending concurrently: every invocation is immediately followed by its matching response or abort.

Reagent semantics. We assume a denotational semantics $\llbracket \cdot \rrbracket$ mapping reagents to partial functions on a state space Σ :

$$\llbracket r \rrbracket : \Sigma \times A \rightarrow \Sigma \times B.$$

$\llbracket r \rrbracket$ is partial because a reagent may *retry* (its precondition is not met at the given state and input). For the primitive reagents:

$$\begin{aligned} \llbracket \text{lift } f \rrbracket(\sigma, a) &= (\sigma, f(a)) \\ \llbracket \text{upd } \ell \ f \rrbracket(\sigma, b) &= (\sigma[\ell \mapsto v'], c) \text{ if } f(\sigma(\ell), b) = \text{Some}(v', c), \text{ undefined otherwise} \\ \llbracket r_1 \gg r_2 \rrbracket(\sigma, a) &= \llbracket r_2 \rrbracket(\sigma', b) \text{ where } (\sigma', b) = \llbracket r_1 \rrbracket(\sigma, a) \\ \llbracket r_1 + r_2 \rrbracket(\sigma, a) &\in \{ \llbracket r_1 \rrbracket(\sigma, a) \} \cup \{ \llbracket r_2 \rrbracket(\sigma, a) \} \text{ if either is defined} \end{aligned}$$

For *swap* we defer to §3.2.

Opacity (Guerraoui–Kapalka). A history H of STM transactions is *opaque* iff there exists a sequential history H_s equivalent to H such that:

1. H_s respects the real-time order of completed transactions in H (if t_1 's response precedes t_2 's invocation in H , then $t_1 \prec t_2$ in H_s);
2. H_s is *legal*: each transaction reads values consistent with running the prefix of committed transactions sequentially;
3. aborted transactions in H_s also see consistent snapshots (i.e. they never observe an intermediate, non-sequential state).

Clause 3 is what distinguishes opacity from strict serialisability; it is what prevents an aborted transaction from observing “impossible” states that could, e.g., cause an infinite loop or segmentation fault in user code.

3 Reagent Opacity

We generalise opacity along two axes.

3.1 Choice

For $r = r_1 + r_2$, a committed response event $\text{res}(r, w)$ is justified if *exactly one* of r_1, r_2 produces w at the linearisation point. Aborted branches must not be observable: if r_1 was tried first and then abandoned, the linearisation must not show writes from r_1 interleaved with committed transactions.

This is automatic under Herlihy–Wing linearisability once we say the linearisation step for $r_1 + r_2$ is *the step of whichever branch committed*. The work is in the implementation, not the specification.

3.2 Rendezvous

Swap is not a unary transaction: it requires a *partner*. Denotationally we model a swap pair as a single joint step:

$$\llbracket \text{swap } ep \mid \text{swap } \overline{ep} \rrbracket(\sigma, (a, c)) = (\sigma, (c, a)),$$

where $\overline{ep}$ is ep 's dual. (For swaps embedded inside larger reagents $r \gg \text{swap } ep \gg r'$, we treat $r \gg r'$ and its dual chain as a joint atomic step — this is the point of reagents.)

A swap appears in the history as two events $\text{inv}(r_1, a)$ and $\text{inv}(r_2, c)$ from different threads. In the linearisation these two events must be *co-linearised*: they occupy a single combined step, with outputs $\text{res}(r_1, c)$ and $\text{res}(r_2, a)$ emerging from that single step.

3.3 The Condition

Definition 1 (Reagent Opacity). *Let H be a history of reagent invocations. H is reagent-opaque iff there exists a sequential history H_s and a co-linearisation map μ on swap events such that:*

1. H_s respects real-time order among completed non-swap events.
2. For each pair of swap events e_1, e_2 with $\mu(e_1) = e_2$, e_1 and e_2 are co-linearised: they are adjacent in H_s and form a single atomic step.
3. H_s is legal: for each committed $\text{run}(r, v) \Rightarrow w$ in H_s with pre-state σ and post-state σ' , $(\sigma', w) \in \llbracket r \rrbracket(\sigma, v)$.
4. For each r of the form $r_1 + r_2$ with committed response w in H_s , the transition $(\sigma, v) \rightarrow (\sigma', w)$ equals $\llbracket r_1 \rrbracket(\sigma, v)$ or $\llbracket r_2 \rrbracket(\sigma, v)$ (and only one).
5. Every aborted/retried event in H observes a state σ_a that is reachable as an intermediate state in H_s : i.e., H_s extended with a virtual aborted transaction reading σ_a is still legal.

Clause 5 is opacity's safety-of-aborts property. Clauses 2 and 4 are reagent-specific.

Remark 1 (Relation to linearisability). *Reagent opacity strictly refines the (library-local) linearisability condition on reagent operations, since it additionally constrains aborts (clause 5) and forbids intermediate states from leaking through retries. Where only committed reactions are observed, reagent opacity degenerates to strict serialisability plus the rendezvous and choice constraints.*

Remark 2 (Relation to Attiya–Gotsman). *The Attiya–Gotsman framework for STM consistency conditions defines client-observable equivalences between an implementation and a sequential specification. Reagent opacity slots into that framework: the sequential specification is given by $\llbracket \cdot \rrbracket$ with a rule pairing swap invocations; the implementation is any concrete reagent runtime (our layered `01_kcas-06_reagent_cps`); and correctness is the existence of a linearisation witnessing reagent opacity. The reagent-specific clauses appear as side conditions on the linearisation map, not as substrate changes.*

4 Proof Obligations for the Layered Implementations

The condition breaks into per-layer obligations.

01_kcas (GKMZ k-CAS). Each `Kcas.atomically` call is a single atomic step on its locations. This is a classical linearisability obligation for k-CAS, discharged by the two-phase protocol (descriptor install; status flip).

03_stm (Xt). The log-based STM on top of k-CAS induces *ordinary* opacity for compositions of `upd`, `read`, `cas`, and `>>`. The log’s `CMP` entries supply the read-validation required by clauses 3 and 5; `Xt.or_else`’s log snapshot restores clause 4’s “only one branch observable” property.

05_lightweight (direct-style swap). The direct-style channel swap lies *outside* the `Xt` log. A history in which a reagent `swap ep >> cas  $\ell$  v v'` commits is **not** in general reagent-opaque under this definition: the swap linearises at the rendezvous (when the partner is fulfilled), but the `cas` linearises later at `Xt.commit`, and an intervening thread can observe the gap. This mismatch is the motivation for the CPS rewrite.

06_reagent_cps (CPS with try_swap). Reagent opacity holds for compositions not involving swap, and for `try_swap + r` at the *branch-selection* level (clause 4) when no partner is present. It does not hold for `try_swap >> r` when a partner has been fulfilled but `r` then retries: in that case the partner observes the rendezvous (clause 2 gives them a completed swap event) while our thread retries, violating clause 3.

A faithful implementation would require the offer/reservation protocol described in [1] §5: the offer slot’s Pending→Fulfilled CAS is part of the same atomic bundle as the downstream reagent’s CASes, so the partner’s linearisation point and ours coincide exactly as demanded by clause 2.

5 Open Questions

- **Fairness.** The condition above is safety-only. It says nothing about liveness: a pair of swap invocations may wait forever for a partner without violating reagent opacity. Is there a liveness companion? Weak progress (some reagent always makes progress) is probably the right level.
- **Nested choice.** Can clause 4 be strengthened to account for $(r_1 + r_2) >> r_3$? The sequence should commit with r_3 -in-the-same-atomic-step, regardless of which branch of the choice won. The condition above handles this through the denotation $\llbracket r_1 >> r_3 \rrbracket \cup \llbracket r_2 >> r_3 \rrbracket$, but a more direct characterisation is desirable.
- **Retries and starvation.** In the minimal CPS implementation, a reagent returning `ShouldRetry` loops with `Sched.yield`. An opacity-preserving implementation should also rule out livelock under adversarial scheduling.
- **Relation to TMS1/TMS2.** Reagent opacity can likely be expressed as an instance of TMS-style spec with custom atomic steps (joint swap, branch projection). Working this out in detail would pin the condition into an established framework.

References

- [1] A. Turon. *Understanding and expressing scalable concurrency*. PhD thesis, Northeastern University, 2013.
- [2] A. Turon. Reagents: expressing and composing fine-grained concurrency. *ICFP 2012*.
- [3] R. Guerraoui and M. Kapalka. On the correctness of transactional memory. *PPoPP 2008*.
- [4] S. Doherty, L. Groves, V. Luchangco, M. Moir, M. Scott. Towards formally specifying and verifying transactional memory. *Formal Aspects of Computing*, 2013.

- [5] H. Attiya and A. Gotsman. Transactional memory: models and algorithms (and client-observable consistency conditions more broadly). See: Koskinen–Parkinson, Lesani–Palsberg, and the Attiya–Hans–Gotsman line on consistency-condition refinements.